API Explorer - Arquitectura y Documentación Técnica

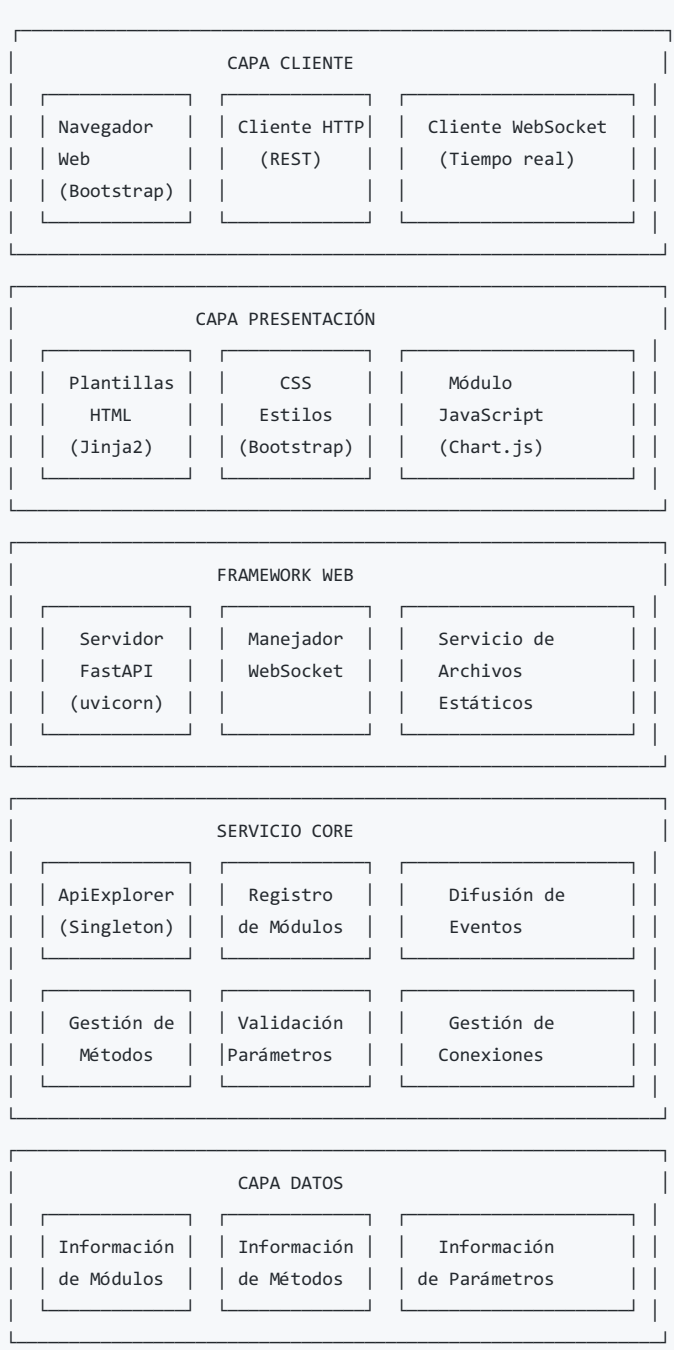
Resumen

API Explorer es un framework web integral diseñado para proporcionar capacidades dinámicas de exploración y prueba de APIs para módulos Python. Construido sobre FastAPI con soporte WebSocket, ofrece una interfaz moderna y responsiva que permite a múltiples módulos registrar sus APIs en tiempo de ejecución, proporcionando formularios de parámetros interactivos, transmisión de eventos en tiempo real, visualización de datos y capacidades de prueba comprehensivas. El sistema implementa un patrón singleton para accesibilidad global mientras mantiene operaciones thread-safe y arquitectura extensible.

Arquitectura General

Patrón Arquitectónico Principal

El API Explorer sigue una **arquitectura de servicio basada en singleton** combinada con patrones **observer** y **registry**, organizando la funcionalidad en capas bien definidas:



Componentes Core y Arquitectura

1. API Explorer Core (`api_explorer.py`)

Propósito: Servicio singleton central que gestiona todo el framework de exploración de APIs.

Componentes:

- `ApiExplorer` : Clase singleton principal que coordina todas las operaciones
- **Registro de Módulos**: Sistema de registro dinámico para módulos externos
- **Gestor WebSocket**: Manejador de comunicación en tiempo real
- **Gestión de Rutas**: Generación dinámica de rutas FastAPI
- **Difusión de Eventos**: Sistema de distribución de eventos multi-cliente

Características Clave:

- **Patrón Singleton**: Instancia única thread-safe con double-checked locking
- **Registro Dinámico**: Registro de módulos y métodos en tiempo de ejecución
- **Operaciones Thread-Safe**: Múltiples locks para thread safety granular
- **Gestión de Puertos**: Detección automática de puertos y asignación
- **Ciclo de Vida del Servidor**: Gestión completa de inicio/parada del servidor

```
# Ejemplo de uso
from api_explorer import ApiExplorer

explorer = ApiExplorer.get_instance()
explorer.register_module("mi_modulo", "Mi Módulo", "Descripción")
explorer.add_method_group("mi_modulo", "control", "Métodos de Control")
explorer.register_method("mi_modulo", "control", "iniciar", method_info, handler)
```

Detalles de Arquitectura:

- **Thread Safety**: Múltiples locks (`_lock` , locks específicos de router)
- **Gestión de Conexiones**: Pool de conexiones WebSocket con seguimiento de ciclo de vida
- **Sistema de Eventos**: Difusión de eventos a múltiples clientes conectados
- **Integración Sistema de Archivos**: Navegación de directorios del lado del servidor y operaciones de archivos
- **Manejo de Errores**: Aislamiento comprehensivo de errores y recuperación

2. Módulo Principal (`main.py`)

Propósito: Punto de entrada y gestión de servidor de alto nivel.

Componentes:

- `run_explorer()` : Función principal para iniciar el API Explorer
- `main()` : Punto de entrada de línea de comandos
- **Configuración del Servidor**: Configuración de host, puerto y bloqueo
- **Gestión de Rutas**: Configuración de rutas Python para importaciones

Características:

- **Despliegue Flexible**: Soporta modos de servidor bloqueante y no bloqueante
- **Gestión de Configuración**: Configuración de host y puerto
- **Gestión de Importaciones**: Resolución automática de rutas para importaciones de módulos
- **Abstracción del Servidor**: Interfaz limpia para integración de módulos externos

3. Arquitectura Frontend

Plantilla HTML (`templates/index.html`)

Propósito: Interfaz web dinámica y responsiva con framework Bootstrap.

Componentes Clave:

- **Interfaz de Registro de Módulos**: Generación dinámica de lista de módulos
- **Pestañas de Explorador API**: Interfaz de exploración API por módulo
- **Paneles de Eventos en Tiempo Real**: Visualización y display de eventos WebSocket
- **Formularios de Parámetros**: Generación dinámica de formularios basada en firmas de métodos
- **Modal de Navegador de Archivos**: Navegación del sistema de archivos del lado del servidor
- **Visualización de Datos**: Integración Chart.js para streaming de datos en tiempo real

Características:

- **Diseño Responsivo**: Layout responsivo basado en Bootstrap
- **Modo Oscuro/Claro**: Sistema completo de temas con persistencia de preferencias de usuario
- **Contenido Dinámico**: Plantillas Jinja2 con actualizaciones impulsadas por JavaScript
- **Actualizaciones en Tiempo Real**: Actualizaciones de contenido en vivo impulsadas por WebSocket
- **Formularios Interactivos**: Validación de parámetros y controles de entrada específicos por tipo

Estilos CSS (`static/css/styles.css`)

Propósito: Sistema de estilos comprehensivo con soporte de tema dual.

Componentes:

- **Estilos Modo Oscuro**: Implementación completa de tema oscuro
- **Estilos Modo Claro**: Tema claro por defecto con apariencia profesional
- **Estilos de Componentes**: Estilos consistentes a través de todos los componentes UI
- **Layout Responsivo**: Principios de diseño responsivo mobile-first
- **Integración de Gráficos**: Integración de temas Chart.js

Características:

- **Consistencia de Tema**: Paleta de colores unificada y tipografía
- **Accesibilidad**: Ratios de alto contraste y consideraciones de cumplimiento WCAG
- **Diseño Moderno**: Principios de diseño web contemporáneo
- **Compatibilidad Cross-browser**: Apariencia consistente entre navegadores

Módulo JavaScript (`static/js/script.js`)

Propósito: Lógica de aplicación del lado del cliente y comunicación en tiempo real.

Componentes Clave:

- **Gestión WebSocket**: Manejo de conexiones WebSocket por módulo
- **Visualización de Gráficos**: Visualización de datos en tiempo real con Chart.js
- **Procesamiento de Eventos**: Manejo de eventos en tiempo real y actualizaciones de UI
- **Gestión de Formularios**: Generación dinámica de formularios y validación
- **Navegador de Archivos**: Interfaz de navegación del sistema de archivos del lado del cliente
- **Gestión de Temas**: Cambio de modo oscuro/claro y persistencia

Características:

- **Arquitectura Dirigida por Eventos**: Actualizaciones de UI reactivas basadas en eventos WebSocket
- **Soporte Multi-Módulo**: Conexiones WebSocket independientes por módulo
- **Gestión de Gráficos**: Múltiples tipos de gráficos con modos de separación de canales
- **Filtrado en Tiempo Real**: Filtrado y display de eventos basado en tipo
- **Manejo de Errores**: Manejo robusto de errores con retroalimentación de usuario
- **Gestión de Estado**: Persistencia de estado del lado del cliente y sincronización

```
// Ejemplo de conexión WebSocket
function connectWebSocket(moduleId) {
    const ws = new WebSocket(`ws://localhost:8000/ws/${moduleId}`);
    ws.onmessage = (event) => {
        const data = JSON.parse(event.data);
        updateUI(moduleId, data);
    };
    webSockets[moduleId] = ws;
}
```

Estructuras de Datos y Modelos

1. ModuleInfo

Propósito: Almacenamiento completo de metadatos y configuración de módulos.

Atributos:

- **Identidad del Módulo**: ID, nombre, descripción y preferencias de display
- **Grupos de Métodos**: Colección organizada de métodos relacionados
- **Integración Router**: Router FastAPI para endpoints HTTP
- **Configuración**: Opciones de display y configuraciones específicas del módulo

Características:

- **Organización Jerárquica**: Métodos organizados en grupos lógicos
- **Actualizaciones Dinámicas**: Modificación en tiempo de ejecución de configuración de módulos
- **Serialización**: Serialización JSON completa para transmisión a cliente

2. MethodInfo

Propósito: Metadatos detallados de métodos y configuración de ejecución.

Atributos:

- **Identidad del Método**: ID, nombre, descripción y categorización
- **Definición de Parámetros**: Especificación completa de parámetros con validación
- **Integración de Manejador**: Vinculación directa a funciones de ejecución
- **Documentación**: Documentación comprehensiva del método

Características:

- **Validación de Parámetros**: Verificación de tipos y aplicación de restricciones
- **Ejecución Dinámica**: Invocación de métodos en tiempo de ejecución con binding de parámetros
- **Manejo de Errores**: Captura y reporte comprehensivo de errores

3. ParameterInfo

Propósito: Especificación comprehensiva de parámetros y validación.

Atributos:

- **Sistema de Tipos**: Soporte para todos los tipos básicos Python más parámetros de archivo
- **Reglas de Validación**: Valores min/max, campos requeridos, validación personalizada
- **Generación UI**: Generación automática de controles de formulario basada en tipo de parámetro
- **Valores por Defecto**: Especificación y manejo de valores por defecto

Características:

- **Soporte de Tipos Rico**: String, número, booleano, elección, archivo, y tipos personalizados
- **Validación Avanzada**: Verificación de rangos, coincidencia de patrones, validadores personalizados
- **Adaptación UI**: Diferentes controles de formulario basados en características de parámetros
- **Integración de Archivos**: Integración de navegador de archivos para parámetros de archivo

```
 # Ejemplo de definición de parámetro
param_info = {
    "type": "float",
    "required": True,
    "description": "Frecuencia de muestreo en Hz",
    "min": 1.0,
    "max": 1000.0,
    "step": 0.1,
    "default": 250.0
}
```

Características y Capacidades Avanzadas

1. Visualización de Datos en Tiempo Real

Integración de Gráficos:

- **Framework Chart.js**: Gráficos profesionales con múltiples tipos de gráficos
- **Streaming en Tiempo Real**: Actualizaciones de datos en vivo vía WebSocket
- **Soporte Multi-canal**: Visualización separada para diferentes canales de datos
- **Integración de Temas**: Temas automáticos de gráficos basados en modo UI

Modos de Visualización:

- **Gráficos Combinados**: Todos los canales en un solo gráfico para comparación
- **Gráficos Separados**: Gráficos individuales por canal para análisis detallado
- **Filtrado por Tipo**: Filtrar display por tipo de datos (EEG, EMG, etc.)
- **Auto-escalado**: Escalado dinámico de ejes basado en rangos de datos

Características:

- **Optimización de Rendimiento**: Buffering eficiente de datos y renderizado
- **Controles Interactivos**: Zoom, pan e inspección de puntos de datos
- **Capacidades de Exportación**: Exportación de gráficos para documentación y análisis
- **Diseño Responsivo**: Los gráficos se adaptan a diferentes tamaños de pantalla

2. Sistema de Eventos WebSocket

Comunicación en Tiempo Real:

- **Canales por Módulo**: Conexiones WebSocket independientes para cada módulo

- **Difusión de Eventos**: Entrega simultánea a múltiples clientes conectados
- **Gestión de Conexiones**: Reconexión automática y monitoreo de heartbeat
- **Cola de Mensajes**: Entrega confiable de mensajes con buffering

Tipos de Eventos:

- **Eventos de Datos**: Streaming de datos en tiempo real para visualización
- **Eventos de Estado**: Actualizaciones de estado de módulos y sistema
- **Eventos de Control**: Confirmaciones de control interactivo y retroalimentación
- **Eventos de Error**: Reporte de errores e información diagnóstica

Características:

- **Escalabilidad**: Soporte para múltiples conexiones simultáneas
- **Confiabilidad**: Monitoreo de conexiones y recuperación automática
- **Seguridad**: Validación de conexiones y control de acceso
- **Rendimiento**: Codificación eficiente de mensajes binarios y JSON

3. Integración del Sistema de Archivos

Navegador de Archivos del Lado del Servidor:

- **Navegación de Directorios**: Navegación completa del árbol de directorios del lado del servidor
- **Filtrado de Archivos**: Filtrado basado en extensiones y capacidades de búsqueda
- **Resolución de Rutas**: Manejo y resolución de rutas cross-platform
- **Seguridad**: Acceso a archivos en sandbox con validación de permisos

Operaciones de Archivos:

- **Selección de Archivos**: Selección interactiva de archivos para entrada de parámetros
- **Entrada de Rutas**: Entrada manual de rutas con validación
- **Modos Guardar/Cargar**: Soporte para operaciones tanto de carga como guardado de archivos
- **Manejo de Prefijos**: Generación automática de prefijos de archivo para guardado

Características:

- **Cross-platform**: Soporte para Windows, macOS y Linux
- **Rendimiento**: Manejo eficiente de directorios grandes
- **Seguridad**: Acceso seguro a archivos con verificaciones de permisos
- **Usabilidad**: Interfaz intuitiva de navegador de archivos

4. Formularios de Parámetros Dinámicos

Generación de Formularios:

- **Controles Basados en Tipos**: Selección automática de controles de formulario basada en tipo de parámetro
- **Integración de Validación**: Validación en tiempo real con retroalimentación visual
- **Controles Avanzados**: Sliders, dropdowns, navegadores de archivos y controles personalizados
- **Gestión de Layout**: Layout de formulario responsivo con espaciado apropiado

Tipos de Parámetros:

- **Tipos Básicos**: String, entero, float, booleano
- **Tipos Avanzados**: Elección (dropdown), rango (slider), archivo
- **Tipos Personalizados**: Tipos de parámetros específicos del módulo con validación personalizada
- **Tipos Complejos**: Parámetros de array y objeto con validación anidada

Características:

- **Validación en Tiempo Real**: Retroalimentación inmediata sobre validez de parámetros
- **Manejo de Errores**: Mensajes de error claros y guía de corrección
- **Valores por Defecto**: Manejo inteligente de valores por defecto y persistencia
- **Accesibilidad**: Navegación completa por teclado y soporte para lectores de pantalla

Patrones de Integración y Uso

1. Patrón de Registro de Módulos

Los módulos se integran con API Explorer a través de un patrón de registro simple:

```
# Registro básico de módulo
explorer = ApiExplorer.get_instance()
explorer.register_module("mi_modulo", "Mi Módulo", "Descripción del módulo")

# Agregar grupos de métodos
explorer.add_method_group("mi_modulo", "control", "Métodos de Control")
explorer.add_method_group("mi_modulo", "datos", "Acceso a Datos")

# Registrar métodos individuales
method_info = {
    "name": "Iniciar Grabación",
    "description": "Comenzar grabación de datos",
    "type": "control",
    "params": {
        "duracion": {
            "type": "float",
            "required": True,
            "description": "Duración de grabación en segundos",
            "min": 1.0,
            "max": 3600.0
        }
    }
}

def start_recording_handler(duracion: float):
    # Implementación
    return {"status": "iniciado", "duracion": duracion}

explorer.register_method("mi_modulo", "control", "iniciar_grabacion",
                         method_info, start_recording_handler)
```

2. Patrón de Difusión de Eventos

Los eventos en tiempo real se difunden a clientes conectados:

```
# Difundiendo eventos a clientes
event_data = {
    "timestamp": time.time(),
    "type": "actualizacion_datos",
    "data": {
        "canal_1": [1.2, 1.5, 1.8],
        "canal_2": [0.8, 0.9, 1.1]
    },
    "visualization": {
        "chart_type": "line",
        "update_mode": "append"
    }
}

explorer.broadcast_event("mi_modulo", event_data)
```

3. Patrón de Integración del Servidor

Integración con aplicaciones existentes:

```
# Iniciar API Explorer junto con aplicación existente
def initialize_api_integration():
    explorer = ApiExplorer.get_instance()

    # Registrar módulo
    setup_module_registration(explorer)

    # Iniciar servidor en hilo separado
    explorer.start_server(host="localhost", port=8000, block=False)

    return explorer

# Cierre limpio
def shutdown_api_integration():
    explorer = ApiExplorer.get_instance()
    explorer.stop_server()
```

Patrones Arquitectónicos Implementados

1. Patrón Singleton

- `ApiExplorer` : Singleton global con instanciación thread-safe
- **Gestión de Estado Global**: Estado consistente a través de todos los componentes
- **Compartición de Recursos**: Conexiones WebSocket compartidas y recursos del servidor

2. Patrón Registry

- **Registro de Módulos**: Registro dinámico de módulos y métodos
- **Registro de Manejadores**: Mapeo de endpoints API a funciones manejadoras
- **Registro de Conexiones**: Gestión de conexiones WebSocket activas

3. Patrón Observer

- **Eventos WebSocket**: Notificación de eventos en tiempo real a múltiples observadores
- **Actualizaciones de Cliente**: Actualizaciones automáticas de UI basadas en eventos del servidor
- **Sincronización de Estado**: Actualizaciones automáticas de estado del cliente

4. Patrón Factory

- **Generación de Formularios**: Creación dinámica de formularios basada en especificaciones de parámetros
- **Creación de Gráficos**: Instanciación de gráficos basada en tipos de datos
- **Generación de Rutas**: Creación dinámica de rutas FastAPI

5. Patrón Adapter

- **Integración de Frameworks**: Adaptación entre diferentes frameworks Python
- **Abstracción WebSocket**: Interfaz unificada para diferentes implementaciones WebSocket
- **Abstracción del Sistema de Archivos**: Operaciones del sistema de archivos cross-platform

6. Patrón Command

- **Ejecución de Métodos API**: Encapsulación de llamadas de métodos con parámetros
- **Manejo de Eventos**: Procesamiento de eventos basado en comandos
- **Soporte Deshacer/Rehacer**: Potencial para historial de comandos y reversión

Características de Rendimiento y Escalabilidad

1. Manejo Eficiente de Datos

- **Streaming**: Streaming de datos en tiempo real sin acumulación de memoria
- **Buffering**: Buffering inteligente para visualización suave
- **Compresión**: Compresión de datos para datasets grandes
- **Caching**: Caching estratégico de datos frecuentemente accedidos

2. Gestión de Conexiones

- **Gestión de Pool**: Pooling eficiente de conexiones WebSocket
- **Limpieza de Recursos**: Limpieza automática de conexiones inactivas
- **Balanceo de Carga**: Distribución de carga a través de múltiples conexiones

- **Monitoreo de Heartbeat**: Monitoreo de salud de conexiones y recuperación

3. Rendimiento de UI

- **Scrolling Virtual**: Manejo eficiente de listas de datos grandes
- **Carga Perezosa**: Carga bajo demanda de componentes UI
- **Optimización de Gráficos**: Renderizado y actualizaciones eficientes de gráficos
- **Gestión de Memoria**: Limpieza automática de recursos UI

Características de Seguridad y Confiabilidad

1. Validación de Entrada

- **Validación de Parámetros**: Validación comprehensiva de todos los parámetros de entrada
- **Prevención de Inyección SQL**: Protección contra ataques de inyección
- **Prevención XSS**: Protección contra cross-site scripting
- **Protección CSRF**: Protección contra cross-site request forgery

2. Manejo de Errores

- **Aislamiento de Excepciones**: Errores en un módulo no afectan a otros
- **Degradación Grácil**: El sistema continúa operando a pesar de fallas de componentes
- **Reporte de Errores**: Logging y reporte comprehensivo de errores
- **Mecanismos de Recuperación**: Recuperación automática de condiciones de error comunes

3. Gestión de Recursos

- **Límites de Memoria**: Protección contra agotamiento de memoria
- **Límites de Conexión**: Límites en conexiones concurrentes
- **Limitación de Tasa**: Protección contra abuso de API
- **Manejo de Timeouts**: Manejo apropiado de operaciones de larga duración

Casos de Uso y Aplicaciones

1. Desarrollo y Prueba de APIs

- **Pruebas Interactivas**: Pruebas de API en tiempo real con ajuste de parámetros
- **Documentación**: Generación automática de documentación de API
- **Depuración**: Capacidades de depuración y diagnóstico en tiempo real
- **Pruebas de Integración**: Pruebas de integraciones API en escenarios realistas

2. Monitoreo de Datos en Tiempo Real

- **Datos de Sensores**: Visualización en tiempo real de streams de datos de sensores
- **Monitoreo de Sistema**: Monitoreo en vivo de métricas y rendimiento del sistema
- **Instrumentos Científicos**: Adquisición y visualización de datos en tiempo real
- **Aplicaciones IoT**: Monitoreo y control de dispositivos IoT

3. Educación e Investigación

- **Educación API**: Ambiente de aprendizaje interactivo para conceptos de API
- **Herramientas de Investigación**: Plataforma para recolección y análisis de datos de investigación
- **Prototipado**: Prototipado rápido de aplicaciones basadas en API
- **Demostración**: Demostración profesional de capacidades de API

4. Herramientas de Desarrollo

- **Integración de Módulos**: Pruebas de integración para sistemas de módulos complejos
- **Gestión de Configuración**: Configuración dinámica de módulos de aplicación
- **Análisis de Rendimiento**: Monitoreo y análisis de rendimiento en tiempo real
- **Aseguramiento de Calidad**: Pruebas automatizadas y validación de APIs

Stack Tecnológico y Dependencias

Tecnologías Backend:

- **FastAPI**: Framework web moderno y de alto rendimiento
- **Uvicorn**: Servidor ASGI ultrarrápido
- **WebSockets**: Comunicación bidireccional en tiempo real
- **Jinja2**: Motor de plantillas poderoso
- **Python 3.8+**: Python moderno con type hints y soporte async

Tecnologías Frontend:

- **Bootstrap 5**: Framework CSS responsivo moderno
- **Chart.js**: Librería de gráficos profesional
- **JavaScript Vanilla**: JavaScript puro para rendimiento óptimo
- **API WebSocket**: Soporte WebSocket nativo del navegador
- **CSS Grid/Flexbox**: Técnicas de layout modernas

Herramientas de Desarrollo:

- **Type Hints**: Anotación de tipos comprehensiva
- **Async/Await**: Programación asíncrona moderna
- **Manejo de Errores**: Gestión comprehensiva de excepciones
- **Logging**: Soporte profesional de logging y depuración

Despliegue y Configuración

1. Despliegue Independiente

```
 # Despliegue independiente simple
from api_explorer import run_explorer

# Iniciar servidor
explorer = run_explorer(host="0.0.0.0", port=8000, block=True)
```

2. Despliegue Integrado

```
 # Integración con aplicación existente
from api_explorer import ApiExplorer

def setup_api_explorer():
    explorer = ApiExplorer.get_instance()
    register_my_module(explorer)
    explorer.start_server(host="localhost", port=8000, block=False)
    return explorer
```

3. Configuración de Producción

- **Proxy Reverso**: Integración Nginx/Apache para despliegue en producción
- **SSL/TLS**: Soporte HTTPS para comunicaciones seguras
- **Variables de Entorno**: Configuración vía variables de entorno
- **Soporte Docker**: Opciones de despliegue containerizado